Median Pixel Filter RTL Project

1. What are we building?

We are building a small hardware module that takes in an image **one pixel at a time** and outputs a new filtered image.

The filter looks at a small 2x2 group of pixels at a time:

Pixel A Pixel B
Pixel C Pixel D

For each group of 4 pixels, we calculate the average color value. Since each pixel has red, green, and blue values, we calculate the median separately for:

Red values
Green values
Blue values

Then we combine those three values back into one output pixel.

This project will give you experience with:

- Buffers
- Functions
- Finite State Machines
- Buffers
- More

P00 **stream #1**	**P01** **stream #2**	P02 stream #3	P03 stream #4	P04 stream #5
P10 **stream #6**	**P11** **stream #7**	P12 stream #8	P13 stream #9	P14 stream #10
P20 stream #11	P21 stream #12	P22 stream #13	P23 stream #14	P24 stream #15
P30 stream #16	P31 stream #17	P32 stream #18	P33 stream #19	P34 stream #20
P40 stream #21	P41 stream #22	P42 stream #23	P43 stream #24	P44 stream #25

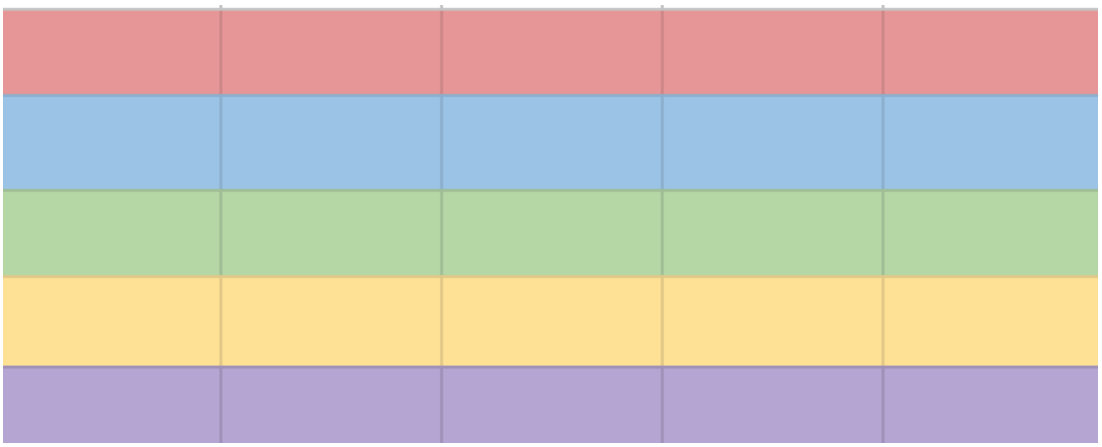
The median filter receives pixels one at a time, stores recent pixels in a buffer, forms a 2x2 window, calculates the median color, and outputs a filtered pixel.

Project

The median pixel filter applies a 2x2 kernel to an image with dimensions IMAGE_LEN and IMAGE_HEIGHT. Since a 2x2 kernel covers an even number of pixels (4 total), there is no single 'middle' pixel. This module calculates a median pixel value by sorting the 4 pixel values and taking the average of the two middle values. This is done for each of the 3 pixel channels (red, green, blue) separately. The resulting median pixel channels should then be combined back into a pixel output.

Understanding the Problem

Picture this grid as a 5x5 image, where every square represents a pixel



The module will receive one pixel at a time when pixel_valid_i = 1. Pixels are streamed in from top left to bottom right. In the image below, the number indicates the order that the pixel is streamed into the module. The 2x2 kernel begins in the top left corner and is indicated by the black box

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

In order to do our first operation we need to grab IMAGE_LEN + KERNEL_SIZE pixels. This means that for a 2x2 kernel with a 5 pixel wide image we need to grab 7 pixels and store them in a buffer. This buffer can just be thought of as an array of pixels. Currently we have pixel 1 at index 0, pixel 7 at index 6, etc. The buffer is visualised below.

1	2	3	4	5
6	7			

After we have done an operation in our buffer, we need to shift a new pixel into the buffer. We do this by moving all the pixels down an index in the buffer, and setting index 6 to be a new pixel. Below is a visual representation of the shifted buffer.

2	3	4	5	6
7	8			

This represents our kernel moving one space to the right along our image.

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

After we shift two more times the kernel should be positioned at the edge of our image, as represented here

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

In this scenario, we need to shift 2 (kernel_len) more times before we can do another operation. These 2 shifts will result in the buffer below.

4	5	6	7	8
9	10			

Repeat this process until you have taken in all the pixels.

Understanding the Convolution Operation

The convolution operation will happen as follows:

1. Extract each of the pixel's color channels
    a. Red: 100, 40, 100, 100
    b. Green: 26, 30, 200, 130
    c. Blue: 120, 120, 21, 20
2. Sort each list of values
    a. Red: 40, 100, 100, 100
    b. Green: 26, 30, 130 , 200
    c. Blue: 20, 21, 120, 120
3. Find the median value of each channel
    a. Red: 100
    b. Green: 80
    c. Blue: 70.5 ↠ rounds up to 71
3. Recombine into a single pixel
    Pixel = {Red: 100, Green: 80, Blue: 71}
4. Output new pixel and set pixel_valid_o = 1

Other Info

Your inputs/outputs will pass this pixel struct, where PIXEL_W = 8.

```
typedef struct {
  logic [PIXEL_W-1:0] red;
  logic [PIXEL_W-1:0] green;
  logic [PIXEL_W-1:0] blue;
} pixel_t;
```

Module I/O

Your module should have the following header format. Changing it will make it a lot harder for automated tests to be run on your design.

```
import pixel_pkg::*;

module median_filter #(
  parameter int IMAGE_LEN    = 1080,
  parameter int IMAGE_HEIGHT = 720
) (
  input  logic   clk,
  input  logic   rst,                // synchronous reset high
  input  logic   start_i,            // starting a new frame
  input  logic   pixel_valid_i,   // incoming pixel
  input  pixel_t pixel_i,

  output logic   done_o,             // filtered the whole image
  output logic   pixel_valid_o,   // output pixel is valid
  output pixel_t pixel_o             // output pixel
);

endmodule
```

Make sure you are adhering to adhering to the coding guide in the repo, a summary is below:

- Registered signals should use _d and _q
- Use snake_case for signal names
- Use SCREAMING_SNAKE_CASE for parameters
- Use always_comb for combinational logic
- Use always_ff @(posedge clk) for sequential logic
- Resets should be synchronous and active high
- Avoid magic numbers

Shifting

Every time a new pixel comes in, the buffer shifts forward.

Ex.

Before new pixel:
1 2 3 4 5 6 7

New pixel arrives: 8

After shift:
2 3 4 5 6 7 8

This allows the 2x2 kernel to move across the image

GitHub Overview

What is GitHub?

GitHub is where we store the code for the project online.

You will use GitHub to:

- Download the starter code
- Save your changes
- Share your code with the team
- Get updates when the project files change

What is Git?

Git is the tool that lets your computer talk to GitHub.

Git lets you:

- Pull code from GitHub
- Save changes
- Push your changes back to GitHub
- Work on your own branch

Project Folder Structure

```
median_filter_project/
├── rtl/
│   └── median_filter.sv
├── tb/
│   └── median_filter_tb.sv
├── pkg/
│   └── pixel_pkg.sv
└── README.md
```

rtl/	contains the actual hardware design
tb/	contains testbench files used to test your design (Explained more in verification training)
pkg/	contains shared definitions, like the pixel_t struct
README	explains what the project is and how to run it

Guide to Using GitHub

What Is GitHub?

GitHub is where we store the code for the project online

You will use GitHub to:

- Download the starter code
- Save your changes
- Share your code with the team
- Get updates when the project files change

What Is Git?

Git is the tool that lets your computer talk to GitHub

Git lets you:

- Pull code from GitHub
- Save your changes
- Push your changes back to GitHub
- Work on your own branch

Step 1: Clone the Repository

Cloning means downloading the project from GitHub onto your computer

```
git clone <repo-link>
```

Then enter the project folder:

```
cd median_filter_project
```

Step 2: Create Your Own Branch

A branch is your own copy of the project where you can make changes without affecting everyone else's code.

```
git checkout -b your-name-median-filter
```

Example:

```
git checkout -b spencer-median-filter
```

Step 3: Work on Your Files

Most of your work will happen in:

```
rtl/median_filter.sv
```

Step 4: Save Your Changes with Git

After editing files, check what changed:

```
git status
```

Add your changes:

```
git add “path_to_updated_file”
```

Commit your changes with a descriptive commit message:

```
git commit -m "Commit message goes here"
```

Example:

```
git commit -m "Completed FSM and median function (untested)"
```

Step 5: Push Your Branch to GitHub

Pushing means uploading your branch to GitHub

```
git push origin your-name-median-filter
```

Step 6: Open a Pull Request

A pull request asks the team to review and merge your changes

On GitHub:

1. Go to the repository
2. Click Pull requests
3. Click New pull request
4. Select your branch
5. Add a short description of what you changed
6. Submit the pull request